

make_function_ref: A More Functional function_ref

Document #: P2472R0

Date: 2021-09-30

Project: Programming Language C++

Audience: Library Evolution Working Group (LEWG)

Reply-to: Jarrad J. Waterloo
<descender76@gmail.com>

Contents

[1 Enhancement](#)

[2 Solution](#)

[3 Acknowledgments](#)

[4 Annex: controversial design decisions](#)

[4.1 auto](#)

[4.2 General Purpose Callable Library](#)

[5 References](#)

Enhancement

"function_ref" **ONLY** has **ONE**, non copy, constructor which takes a const reference to a functor. The "erased_object/obj_" points to the functor and the "erased_function/callback_" points to a binding function that calls the functor's call operator receiving not only the operator parameters but also the "erased_object/obj_". "function_ref" can be created by binding a functor/stateful lambda, a C function pointer/stateless lambda and a C++ member function pointer. There are important differences between these 3 use cases. The functor/stateful lambda use case supports type erased state while the C function pointer/stateless lambda and a C++ member function pointer use cases do not. In order to type erase one parameter of a free function or the this parameter of a member function a functor/stateful lambda must be created to hold the reference to the desired state. This exposes some of the limitations of the current function_ref's proposal. "std::array" is to the C array AS function_ref is to the C function pointer. function_ref is tedious when it comes to being to assign functional things of ~exact function signature while C function pointers are not. Usage of NON typed erased [member] functions is easier to construct than using type erased [member] functions. Type erased [member] functions is generally more common and definitely more useful than their NON type erased counterparts.

Consider for a moment the use case of type erased member functions. This type of single function runtime polymorphism is found throughout the OOP programming world. For instance, in the .NET world, such as C#, has [single and multi cast delegates](#). Embarcadero C++'s has it via their [closure](#). This has also been peddled in the standard C++ world via articles such as [Member Function Pointers and the Fastest Possible C++ Delegates](#), [The Impossibly Fast C++ Delegate](#) and [Impossibly fast delegate in C++11](#). One thing all of these implementations have in common is the ability to easily create these delegates via a constructor with two parameters; the instance to be type erased and the member function pointer.

Similarly too, the use case of type erased functions has been instrumental in the functional programming world for decades in C and other programming languages. The utility of simplifying the construction of these common use cases are demonstrated below.

```
#include <iostream>
// #include <https://raw.githubusercontent.com/TartanLlama/function_ref/master/include/tl/function_ref.hpp>
#include <https://raw.githubusercontent.com/descender76/cpp_proposals/main/function_ref/function_ref_prime.hpp>

void third_party_lib_function1(tl::function_ref<void(bool, int, float)> callback) {
    callback(true, 11, 3.1459f);
}

double third_party_lib_function2(tl::function_ref<double(int, float, bool)> callback) {
    return callback(11, 3.1459f, true);
}

void third_party_lib_function3(tl::function_ref<void(bool, int, float) noexcept> callback) {
    callback(true, 11, 3.1459f);
}

double third_party_lib_function4(tl::function_ref<double(int, float, bool) noexcept> callback) {
    return callback(11, 3.1459f, true);
}

struct bar {
    void baz(bool b, int i, float f)
```

```

{
    std::cout << "bar::baz" << std::endl;
}
double buz(int i, float f, bool b)
{
    std::cout << "bar::buz" << std::endl;
    return i + f + (b ? 1 : 0);
}
void baznoe(bool b, int i, float f) noexcept
{
    std::cout << "bar::baznoe" << std::endl;
}
double buznoe(int i, float f, bool b) noexcept
{
    std::cout << "bar::buznoe" << std::endl;
    return i + f + (b ? 1 : 0);
}
void caz(bool b, int i, float f)
{
    std::cout << "bar::caz" << std::endl;
}
void caz(bool b, int i, float f) const
{
    std::cout << "bar::caz const" << std::endl;
}
}

};

void application_function_ref(tl::function_ref<void(bar&, bool, int, float)> callback) {
    bar b;
    callback(b, true, 11, 3.1459f);
}

void application_function_pointer(tl::function_ref<void(bar*, bool, int, float)> callback) {
    bar b;
    callback(&b, true, 11, 3.1459f);
}

void application_function_value(tl::function_ref<void(bar, bool, int, float)> callback) {
    bar b;
    callback(b, true, 11, 3.1459f);
}

void free_baz1(bool b, int i, float f)
{
    std::cout << "free_baz1" << std::endl;
}

void free_bar_baz1(bar&, bool b, int i, float f)
{
    std::cout << "free_bar_baz1" << std::endl;
}

void free_bar_baz2(const bar&, bool b, int i, float f)
{
    std::cout << "free_bar_baz2" << std::endl;
}

void free_bar_baz3(bar*, bool b, int i, float f)
{
    std::cout << "free_bar_baz3" << std::endl;
}

void free_bar_baz4(const bar*, bool b, int i, float f)
{
    std::cout << "free_bar_baz4" << std::endl;
}

void free_baz2(bool b, int i, float f) noexcept
{
    std::cout << "free_baz2" << std::endl;
}

void free_bar_baz5(bar&, bool b, int i, float f) noexcept
{
    std::cout << "free_bar_baz5" << std::endl;
}

void free_bar_baz6(const bar&, bool b, int i, float f) noexcept
{
    std::cout << "free_bar_baz6" << std::endl;
}

void free_bar_baz7(bar*, bool b, int i, float f) noexcept
{
    std::cout << "free_bar_baz7" << std::endl;
}

```

```

void free_bar_baz8(const bar*, bool b, int i, float f) noexcept
{
    std::cout << "free_bar_baz8" << std::endl;
}

int main()
{
    bar b;
    // member function with type erased state: b
    third_party_lib_function1([&b](bool b1, int i, float f){b.baz(b1, i, f)});
    third_party_lib_function1([&b](auto... args){b.baz(args...)});
    std::cout << third_party_lib_function2([&b](int i, float f, bool b1){return b.buz(i, f, b1)}); << std::endl;
    std::cout << third_party_lib_function2([&b](auto... args){return b.buz(args...)}); << std::endl;
    // member function with type erasure usecase
    // i.e. delegate/closure/OOP callback/event
    third_party_lib_function1(tl::make_function_ref<&bar::baz>(b));
    std::cout << third_party_lib_function2(tl::make_function_ref<&bar::buz>(b)) << std::endl;
    third_party_lib_function3(tl::make_function_ref<&bar::baznoe>(b));
    std::cout << third_party_lib_function4(tl::make_function_ref<&bar::buznoe>(b)) << std::endl;
    // member function without type erasure usecases
    // i.e. unified function pointer
    application_function_ref(tl::make_function_ref<&bar::baz>());
    application_function_ref(tl::make_function_ref<&bar::baz, tl::ref>());
    application_function_pointer(tl::make_function_ref<&bar::baz, tl::pointer>());
    application_function_value(tl::make_function_ref<&bar::baz, tl::value>());
    application_function_ref(tl::make_function_ref<&bar::baznoe>());
    application_function_ref(tl::make_function_ref<&bar::baznoe, tl::ref>());
    application_function_pointer(tl::make_function_ref<&bar::baznoe, tl::pointer>());
    application_function_value(tl::make_function_ref<&bar::baznoe, tl::value>());
    // function without type erasure usecase
    // i.e. C callback without user data
    third_party_lib_function1(tl::make_function_ref<free_baz1>());
    third_party_lib_function1(tl::make_function_ref<static_cast<void(*)>(bool b, int i, float f)>(free_baz1)>()); // explicit
    third_party_lib_function1(tl::make_function_ref<free_baz2>());
    third_party_lib_function1(tl::make_function_ref<static_cast<void(*)>(bool b, int i, float f)>(free_baz2)>()); // explicit
    // function with type erasure usecase
    // i.e. C callback with user data
    third_party_lib_function1(tl::make_function_ref<free_bar_baz1>(b));
    third_party_lib_function1(tl::make_function_ref<free_bar_baz2>(b));
    third_party_lib_function1(tl::make_function_ref<free_bar_baz3>(b));
    third_party_lib_function1(tl::make_function_ref<free_bar_baz4>(b));
    third_party_lib_function3(tl::make_function_ref<free_bar_baz5>(b));
    third_party_lib_function1(tl::make_function_ref<free_bar_baz6>(b));
    third_party_lib_function1(tl::make_function_ref<free_bar_baz7>(b));
    third_party_lib_function1(tl::make_function_ref<free_bar_baz8>(b));
}

```

Solution

In order to make this possible, multiple overloaded make_function_ref functions need to be added. All of these functions delegate to a new public constructor that exposes the two inner pointers of the function_ref implementation.

```

template <class F> class function_ref;

/// Specializations for function types.
template <class R, class... Args> class function_ref<R(Args...)> {
public:
    function_ref(void* obj_, R (*callback_)(void*,Args...)) noexcept : obj_{obj_}, callback_{callback_} {}
private:
    void *obj_ = nullptr;
    R (*callback_)(void *, Args...) = nullptr;
};

template <class R, class... Args> class function_ref<R(Args...) noexcept> {
public:
    function_ref(void* obj_, R (*callback_)(void*,Args...)) noexcept : obj_{obj_}, callback_{callback_} {}
private:
    void *obj_ = nullptr;
    R (*callback_)(void *, Args...) noexcept = nullptr;
};

template<auto mf, typename T> requires std::is_member_function_pointer<decltype(mf)>::value
auto make_function_ref(T& obj);

template<auto mf, typename T> requires std::is_member_function_pointer<decltype(mf)>::value
auto make_function_ref(const T& obj);

template<auto mf> requires std::is_member_function_pointer<decltype(mf)>::value
auto make_function_ref();

class ref {};
class pointer {};
class value {};

template<auto mf, typename T> requires std::is_member_function_pointer<decltype(mf)>::value && std::is_same<T, ref>::value

```

```

auto make_function_ref();

template<auto mf, typename T> requires std::is_member_function_pointer<decltype(mf)>::value && std::is_same<T, pointer>::value
auto make_function_ref();

template<auto mf, typename T> requires std::is_member_function_pointer<decltype(mf)>::value && std::is_same<T, value>::value
auto make_function_ref();

template<typename testType>
struct is_function_pointer
{
    static const bool value =
        std::is_pointer<testType>::value ?
        std::is_function<typename std::remove_pointer<testType>::type>::value :
        false;
};

template<auto f, typename T> requires is_function_pointer<decltype(f)>::value
auto make_function_ref(T& obj);

template<auto f, typename T> requires is_function_pointer<decltype(f)>::value
auto make_function_ref(const T& obj);

template<auto f> requires is_function_pointer<decltype(f)>::value
auto make_function_ref();

```

Acknowledgments

Thanks to Arthur O'Dwyer and Tomasz Kamiński for providing very valuable feedback on earlier drafts of this proposal.

Annex: controversial design decisions

Why not just use [&b](auto... args){b.baz(args...);}?

While succinct, this does have some undesirable consequences.

- It is more complex; at least cognitively speaking. The only pieces the end user really cares about is either "baz" or "b.baz". The capture, the parameter propagation even with auto and the occasional return are all superfluous.
- If this was a C function pointer would the end user be happy with that versus just assigning with "= some_function". In this case the public wrapper interface is more complicated than the internals. Shouldn't that be reversed i.e. abstraction.
- Using a functor or stateful lambda does require creating a object and type even if anonymous that has a lifetime that must be managed making things more complicated.
- Construction which should be one step has now been broken into two which is more cumbersome; see the next decision.

Why not just use bind_front and turn the remaining make_function_ref implementations into a general purpose functors library?

I actually agree that C++ could use a separate function transformation library for the general cases but I don't think that should negate function_ref supporting compatible signatures; i.e. first class support. Such a general purpose library would turn what should be a one step construction process into a two step process; first create a functor and then assign it to the function_ref. Personally I prefer one step and here is why. function_ref is to function pointer, in C, as std::array is to C/C++ array. It is more intuitive/familiar to the end user. a function pointer is simply assigned for compatible/comparable signatures and the end user must create a new function for incompatible signatures in order to do conversions or reordering of parameters and in the case of functors, adding more parameters. This two phase construction is more verbose. make_function_ref is less so and the same name regardless of what is being assigned. It is also similar to make_pair, make_shared_ptr and the other single step make functions; again familiar. I would hesitate to say that end users expect a simple construction methodology based on the prevalence of simpler solutions in other languages and there is no good reason why we should make things deliberately harder especially for common cases when it is not needed to be so.

It should also be noted that there are current C++ limitations that restrict our implementation choices.

- C++ template constructors do not support explicit template parameters meaning having to resort to tag classes which make things more verbose.
- Template functions do not support partial specialization meaning such things that should simply be performed by functions has to be performed by whole partially specialized classes making things more verbose.
- Making heavily templated make_function_ref functions into FRIEND functions of function_ref is borderline impossible, if not so. This is why I still need my public double pointer constructor. I am still for having this constructor anyway for advance end users to prototype other compatible signatures for operators and other function like constructs in C++ that currently don't have a natural binding.

At present, my current make_function_ref solution seems to provide a very concise single step construction that enhances function_ref to support better the 4 use cases: [non] type erased [member/free] functions.

References

[P0792R5] function_ref: a non-owning reference to a Callable
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0792r5.html>

function_ref

https://github.com/TartanLlama/function_ref

Using Delegates (C# Programming Guide)

<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates/using-delegates>

__closure

<http://docwiki.embarcadero.com/RADStudio/Sydney/en/Closure>

Member Function Pointers and the Fastest Possible C++ Delegates

<https://www.codeproject.com/Articles/7150/Member-Function-Pointers-and-the-Fastest-Possible>

The Impossibly Fast C++ Delegates

<https://www.codeproject.com/Articles/11015/The-Impossibly-Fast-C-Delegates>

Impossibly fast delegate in C++11

<https://codereview.stackexchange.com/questions/14730/impossibly-fast-delegate-in-c11>

Dyno: Runtime polymorphism done right

<https://github.com/ldionne/dyno>

[Boost::ext].TE

<https://github.com/boost-ext/te>